Reinforcement Learning Assignment Solutions

Case Study 1: Warehouse Robot Path Optimization

State: Robot position (row, column) in a 10×10 grid.

Actions: Up, Down, Left, Right.

Rewards: Destination +100, Obstacle -50, Valid move -1, Boundary -10.

Goal: Learn the shortest safe path while minimizing battery usage.

Case Study 2: Self-Driving Car

State: Position, speed, traffic signal, nearby vehicles, pedestrians.

Actions: Accelerate, Brake, Left, Right, Straight.

Rewards: Destination +200, Follow rules +20, Collision -100, Red light -50, Delay -1/sec.

Poor reward design may cause unsafe behavior. Exploration helps the agent discover better strategies. Major challenges include large state spaces, delayed rewards, safety constraints, and dynamic traffic.

Case Study 3: Airline Dynamic Pricing

States: Available seats, days to departure, demand, competitor prices.

Actions: Increase price by 5%, Decrease price by 5%, Keep price unchanged.

RL is more suitable because pricing decisions influence future rewards and adapt over time.

Exploration Strategy: ε-Greedy.

Evaluation: Revenue, occupancy, ticket sales, and profit.

Coding Exercise (Python Q-Learning)

```
import numpy as np
import random
import matplotlib.pyplot as plt

GOAL=10
START=0
LEFT=0
RIGHT=1
alpha=0.1
gamma=0.9
epsilon=0.2
episodes=500

Q=np.zeros((GOAL+1,2))
reward_history=[]

for episode in range(episodes):
    state=START
    total_reward=0
    while state!=GOAL:
        if random.random()<epsilon:
            action=random.choice([LEFT,RIGHT])
        else:
            action=np.argmax(Q[state])

        next_state=max(0,state-1) if action==LEFT else min(GOAL,state+1)
        reward=10 if next_state==GOAL else -1

        Q[state,action]+=alpha*(reward+gamma*np.max(Q[next_state])-Q[state,action])
        state=next_state
        total_reward+=reward

    reward_history.append(total_reward)

print(Q)

state=START
path=[state]
while state!=GOAL:
    action=np.argmax(Q[state])
```

```
    state=max(0,state-1) if action==LEFT else min(GOAL,state+1)
    path.append(state)

print(path)

plt.plot(reward_history)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.show()
```

Output

```
Sample Q-Table (values may vary)

[[2.1 4.3]
 [3.5 5.7]
 ...
]

Optimal Path

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Reward Plot

A line graph showing reward per episode gradually improving as the agent learns the optimal policy.
```